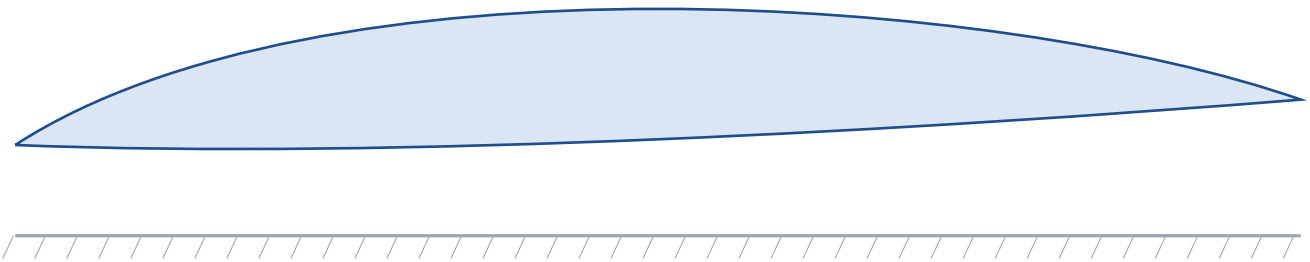
Wing Section Studio

User & Reference Guide

Designing multi-element wing sections in ground effect — configuration, analysis, target-downforce optimization, airfoil shape refinement, manufacturing preparation, and CAD-ready export — together with a complete account of how the aerodynamic engine works.

Part I — Using the application · Part II — How it works
Revision July 2026



Contents

PART I

Using the application

1 Overview

Wing Section Studio designs the two-dimensional section of a multi-element racing wing operating in ground effect — the front wing of a Formula-style car being the canonical case. It covers the fast, iterative phase of wing design: choosing airfoils, arranging one to four elements with correct slot geometry, predicting downforce and drag at the operating point, optimizing the arrangement (and, if you wish, the airfoil shapes themselves) to a target load, preparing the section for manufacture, and exporting CAD-ready geometry.

The application is a screening and optimization layer, not a replacement for high-fidelity analysis. Its solver produces exact inviscid solutions of the full multi-element interaction in milliseconds, corrected to realistic values by transparent, user-adjustable calibration factors. Designs shortlisted here are meant to be confirmed with RANS CFD or wind-tunnel measurement, and the calibration factors re-tuned against those results. Part II of this guide explains every step of that computation.

Capabilities at a glance

- One to four elements; 2,174 bundled UIUC airfoils, 4-digit NACA generation, and Selig `.dat` upload.
- Slot geometry set directly as gap and overlap (percent of chord); placement is solved so the achieved values equal the requested ones.
- Coupled inviscid analysis of all elements in free air and in ground effect, with per-element viscous loading limits and a realistic profile-plus-induced drag estimate.
- Target-downforce optimization over stack angle, deflections, slot geometry, chords, airfoil choice, and airfoil shape — returning a set of distinct candidate designs.
- Manufacturing preparation: buildable trailing edges and thickness checks, folded through the whole pipeline so what you analyze is what you cut.
- Operating maps sweeping ride height or speed around the design point, with the current point and the downforce peak marked.
- Export to DXF, Selig .dat, XYZ points, CSV, SVG, and a JSON manifest, in both the installed and upright orientations — plus a ready-to-run OpenFOAM 2D RANS case.
- Verify a finished design against genuine RANS CFD without leaving the application: one click runs the OpenFOAM case in a local Docker container, watches it converge, compares the result with the estimate, renders the flow field, and offers the calibration factor that reconciles the two.

2 The workflow at a glance

Analysis pipeline (analysis.analyze)

Figure 1 — Inside a single analysis: validated inputs become a placed, solved stack, an inviscid pair of solutions, a corrected estimate, and viscous loading and drag — described in full in Part II.

A typical session runs: pick a starting template; set the operating point (speed, ride height, chord, span); adjust elements and slots while watching the live drawing and loading meters; press **Analyze** to get forces; optimize to a target; optionally refine the airfoil shapes; enable manufacturing preparation; and export. Analysis takes well under a second, so the intended rhythm is conversational — adjust, analyze, repeat.

3 Getting started

Launch by double-clicking **Wing Section Studio** (desktop shortcut or project folder). A window opens immediately with a loading screen while the aerodynamic models start — a few seconds — then the workspace appears. Everything runs locally in one process; closing the window shuts it down. The last session's configuration is saved automatically and restored on the next launch.

Configuration	Operating point, target downforce, the elements, the manufacturing group, and an advanced group of air properties and model-calibration factors.
Elements	One card per element: airfoil, chord, deflection, slot gap and overlap, with live Reynolds number and as-built badges.
Section drawing	A dimensioned technical drawing in the as-driven orientation, with ground hatching, ride-height and length dimensions, slot callouts, and the centre-of-pressure mark. Scroll to zoom, drag to pan.
Results	Estimated downforce against target, coefficient and drag tiles, per-element loading meters, and plain-language warnings.
Detail tabs	Pressure distributions, section polars, the optimizer, the airfoil screener, operating maps, RANS verification, and export.

> **One drawing orientation: as driven**
>
> The drawing always shows the wing the way it runs on the car — inverted, above the road — because that is the orientation the analysis uses. CAD output is a separate choice: the Export tab writes files in either the installed frame or the upright convention airfoil catalogs and CAD sketches use. Both are exact rigid transforms of the same section; nothing about the physics changes.

4 Step 1 — Configure the section

Operating point

Speed	Design airspeed. Forces scale with dynamic pressure; each element's Reynolds number follows from speed and its chord.
Ride height	Clearance from the road to the lowest point of the section, in millimeters — the dominant ground-effect parameter.
Main chord / Span	Main-element chord and wing span. Span enters the reference area and the induced-drag aspect ratio.
Stack angle	Incidence of the whole section. Positive is nose-up — more downforce — the same sense as flap deflection.
Ncrit	Transition criterion for the viscous model. 9 is a clean tunnel; 4–7 represents on-track turbulence.
Air properties & calibration	Density, viscosity, the two calibration factors of the estimate — the free-air viscous realization and the ground-gain realization factor, left blank for the automatic ride-height curve (section 14) — the finite-span lift efficiency, the span efficiency for induced drag, and panel resolution.

Elements and slot geometry

Each element card selects an airfoil — type to search the library, press `.dat` to load a coordinate file — and, for flaps, sets chord (percent of main chord), deflection (degrees, trailing edge down), and the two slot parameters used throughout the high-lift literature:

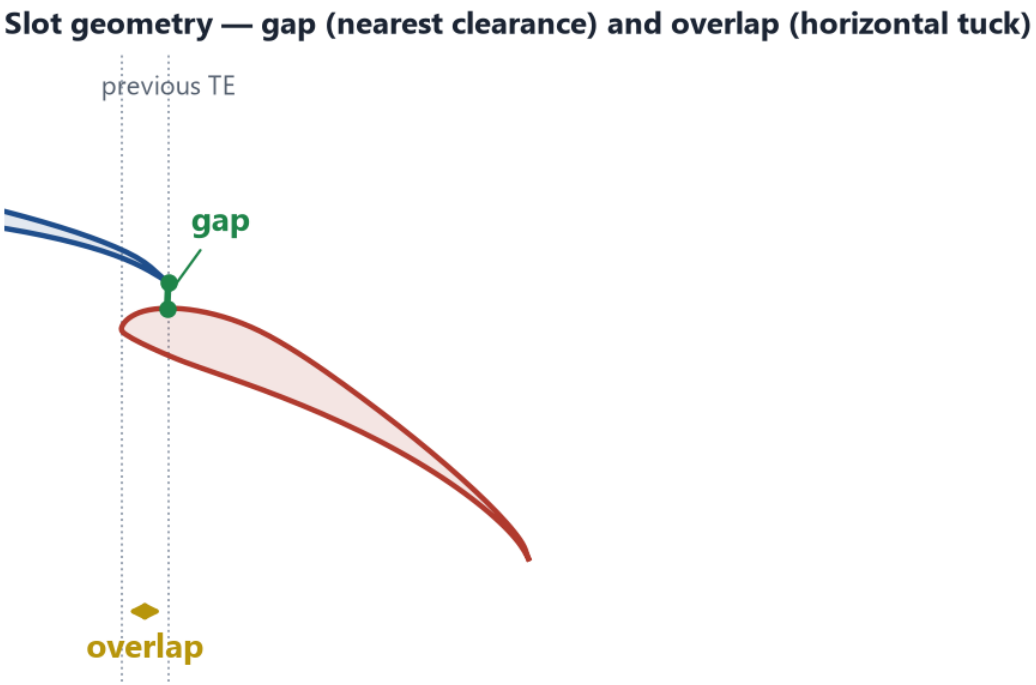


Figure 2 — Slot definitions on a real placement. Gap is the smallest clearance from the previous trailing edge to the flap surface; overlap is the horizontal tuck of the flap nose under that trailing edge.

Placement is solved so the section achieves exactly the requested gap and overlap. Healthy starting ranges are 1–2.5 %c gap and 2–4 %c overlap; the card badges always show the achieved values, and the analysis warns when a slot chokes (gap below 0.5 %c) or a requested gap cannot be reached. The drawing updates live with every edit.

Competition rules as a live envelope

The **Rules** group turns a season's geometric rulebook into an envelope the whole app enforces: maximum installed length, maximum height above the road, minimum ground clearance — all user-entered in mm (rules change every season, so nothing is hardcoded), savable as named presets on this machine ("FSAE 2026") while the active values travel with the project. The drawing shows the envelope as a dashed box and turns a violated

edge amber with the overshoot in mm; the analysis lists the same violation; and the optimizer treats the envelope as hard law — it refuses to start from a violating design and marks out-of-envelope evaluations infeasible, so it can never propose an illegal wing.

5 Step 2 — Analyze

Analyze section solves the coupled inviscid flow around all elements twice — once in free air and once in ground effect — and combines the results with per-element viscous data into the numbers on the right-hand panel:

- **Estimated downforce**, compared against the target as an on-target / over / under pill. The small information button beside the label opens the exact computation.
- **Coefficient tiles** — the corrected estimate alongside the raw inviscid coefficients (ground and free air) it derives from, and the inviscid ground-effect gain.
- **Drag estimate** — induced plus profile, with the split shown beneath the total and the induced-drag details on hover.
- **Element loading meters** — each element's working lift against the isolated maximum of its airfoil at its own Reynolds number. Warnings begin at 90 % of the limit, criticals above 110 %: the classical budget for slotted high-lift systems. A second check watches the realized ground-effect operating point (section 16).
- **Pressure tab** — the inviscid surface-pressure distribution of every element, the physics-level view of how the elements share load.

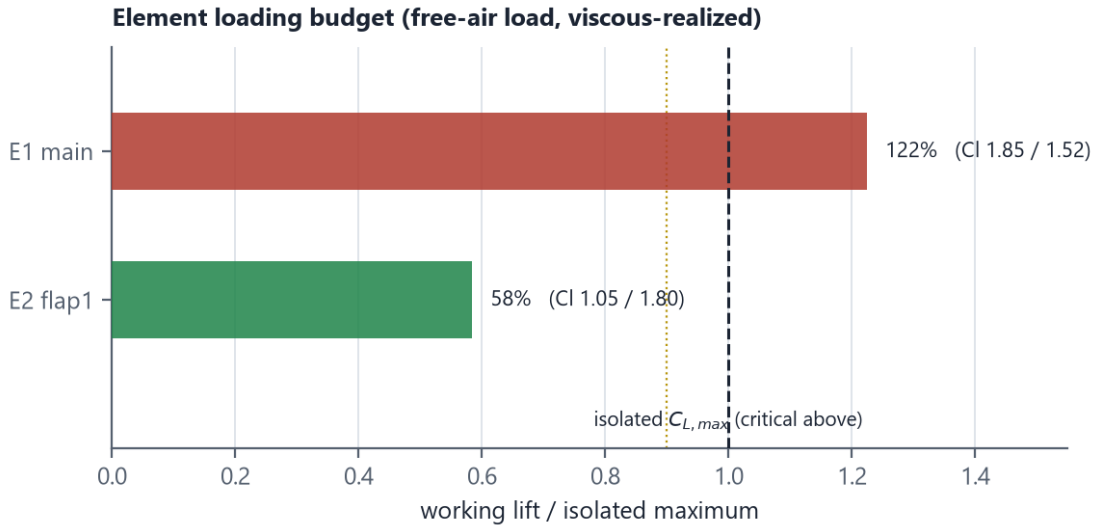


Figure 3 — Loading meters from a real analysis. The main element runs above its isolated maximum (critical, red) — normal for a driven main plane — while the flap sits comfortably under (green). The dotted line is the 90 % warning threshold.

6 Step 3 — Optimize to a target

Set the target downforce and choose the free variables: stack angle, flap deflections, slot gap and overlap, flap chords, the airfoil of each element, and the airfoil shape (section 7). The search drives predicted downforce to the target while penalizing drag, slot geometry outside the healthy band, element overload, intersecting geometry, and designs that lean on viscous data the model itself distrusts — low surrogate confidence or operating points at the edge of the stall data (Part II, The optimizer).

Two goals

Hit a downforce target	The default: track the requested downforce, then spend a reserved share of the budget descending drag along the achievable level. A target below what the stack can cleanly shed to (or beyond its ceiling) is measured, delivered at minimum drag, and reported honestly — the search never fakes an unreachable number by choking its own slots.

Maximize downforce (trusted)	The most downforce the model can be trusted about: every element is held inside the 90 % loading line where fine-mesh RANS measured over-claims of 23–42 %, and the winner and every candidate are re-verified at full resolution before they are returned. An optional minimum L/D keeps the ratio usable, and the user-set confidence floor drops candidates the viscous surrogate itself distrusts.

If manufacturing preparation (section 8) is off when you press **Start optimization**, the application asks first: with prep off, the search would tune knife-edge trailing edges whose flow changes once the wing is made buildable. **Enable & optimize** turns prep on with the defaults and runs on the as-built shapes; **Optimize anyway** runs sharp and is remembered for the rest of the session.

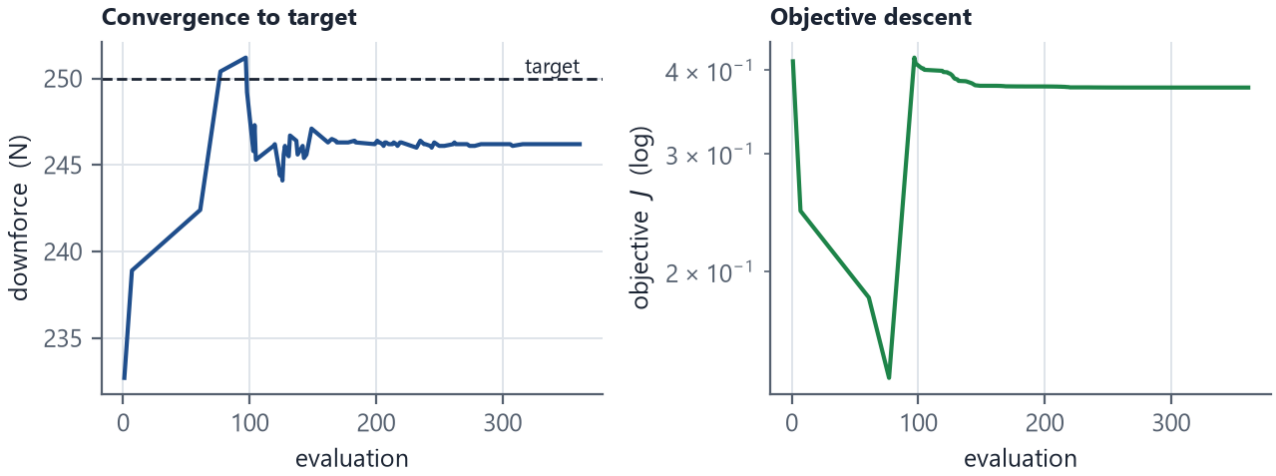


Figure 4 — A real optimization run: downforce converging to the 250 N target (left) and the objective descending on a log scale (right).

Candidates, not a single answer

A finished run returns the best design plus up to five **candidate designs** — distinct set-ups that also hit the target, chosen to be genuinely different from one another (different slot geometry, incidence, or airfoils) rather than trivial variations around one optimum. Each candidate card shows its own analyzed downforce, drag, lift-to-drag, and the minimum surrogate confidence across its sections, and applies to the configuration with one click — so build-tolerance and packaging trade-offs stay visible instead of being hidden inside a single number.

Cards can also carry two trust badges. **Near stall** marks a winner whose loaded elements operate at the edge of their viscous data; **low confidence** marks one resting on sections the surrogate itself distrusts. Both say the same thing: verify with RANS before building. The badges were validated against fine-mesh RANS on the baseline section — an unflagged winner over-claimed by about 14 %, a near-stall-flagged winner by 42 % (and the flagged design, claiming 40 % more downforce, actually delivered less).

Search modes

Global (explore, then polish)	Explores the whole variable space with an evolutionary search before refining the best find — use it to discover the best design for a target, wherever it lives.
Refine current design	Skips the exploration and improves the configured design from where it stands. Quicker, and it stays in the same design basin — the right tool after small manual edits, or to re-tighten a design toward its target.

Either way, the starting design is always reachable: the search bounds and the health penalties widen automatically to include the configuration you started from, so optimizing an already-aggressive design can only improve on it, never hand back less than you had (Part II, The optimizer).

Effort and reproducibility

Fast / Standard	Stop the exploration once the target is reached cleanly — the remaining budget goes to the drag-descend refinement, so even a fast run ends on a low-drag on-target design rather than the first one that hit the number.
Thorough	Searches the whole space before refining, with a wider population and a multi-start final polish at full solver resolution. It takes minutes, and repeated runs converge to the same lowest-drag design — use it for a design you intend to keep.

When a target is unrealistic in either direction, the run measures the stack's clean floor or ceiling and quotes it in plain language — "the lowest clean downforce this stack can make is about 66 N" — and the winner delivers that level at minimum drag instead of a sabotaged number. **Apply to configuration** writes the chosen design, including any airfoil or shape changes, back into the element cards and re-analyzes it.

The trade-off front and the RANS re-rank

Every clean design the search evaluated is kept as a **Pareto front**: the downforce–drag trade-off drawn as a clickable curve, re-analyzed at full resolution — blue where the model is trusted, amber where a point carries a trust flag. Click a point to apply that design. For the designs you would actually build, **RANS re-rank** runs the winner, the candidates and the front's knee through the same 2D RANS truth case one at a time (medium mesh by default — it agreed with the fine mesh in the calibration campaign), re-ranks them by MEASURED downforce, and classes each panel-vs-RANS delta against the recorded bands. The panel model navigates; RANS measures.

7 Step 4 — Airfoil shape refinement

Airfoil selection chooses the best existing section for each element. **Airfoil shape** goes further: it lets the optimizer modify that section — pushing camber where the loading wants it, thinning or thickening the envelope — to squeeze out more performance. Enable it as a free variable alongside the others.

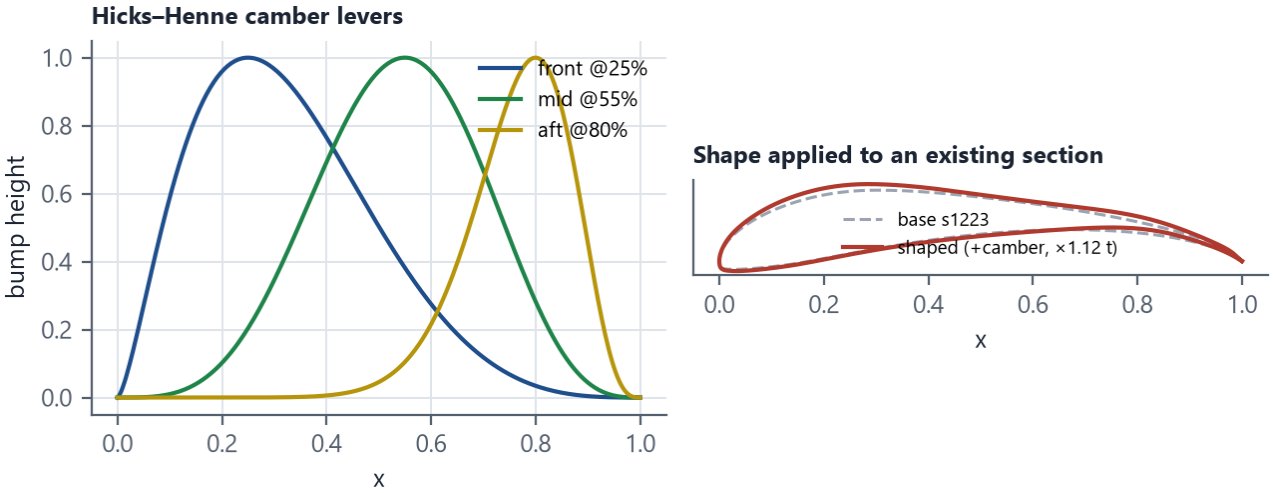


Figure 5 — Shape refinement. Three camber levers (front, mid, aft loading) plus a thickness scale (left) modify whatever section the element uses (right); the modification is applied to a real S1223 here.

Each element gets three camber adjustments (near 25, 55, and 80 % chord — the classical front-, mid-, and aft-loading levers) and one thickness scale. Every constraint still holds: manufacturing trailing-edge preparation wraps the modified section, the buildable-thickness floor limits how far it may be thinned, and the stall budget uses the modified section's own polar. Shaped sections carry through polars, exports, and saved projects; re-running the optimizer keeps refining the current shape rather than stacking changes.

> **Shape refinement is slower**
>
> Every new shape needs fresh aerodynamic evaluation (there is no library polar to reuse), so shape runs are somewhat slower per evaluation — roughly 1.3–1.7× — than geometry-only runs. Pair it with the **Thorough** effort setting when you want a final, reproducible shape, and expect a run to take a few minutes.

8 Step 5 — Manufacturing preparation

Theoretical airfoil coordinates close to a knife edge at the trailing edge — zero thickness. No layup, print, or hot-wire cut can produce that. Enable **Manufacturing** to open every element's trailing edge to a minimum buildable thickness in millimeters, and to check that each section is thick enough to make. The optimizer stands guard on this too: starting a run with prep off raises the prompt described in section 6, so a search never silently tunes geometry the workshop cannot produce.

Trailing-edge treatments — a knife edge cannot be built

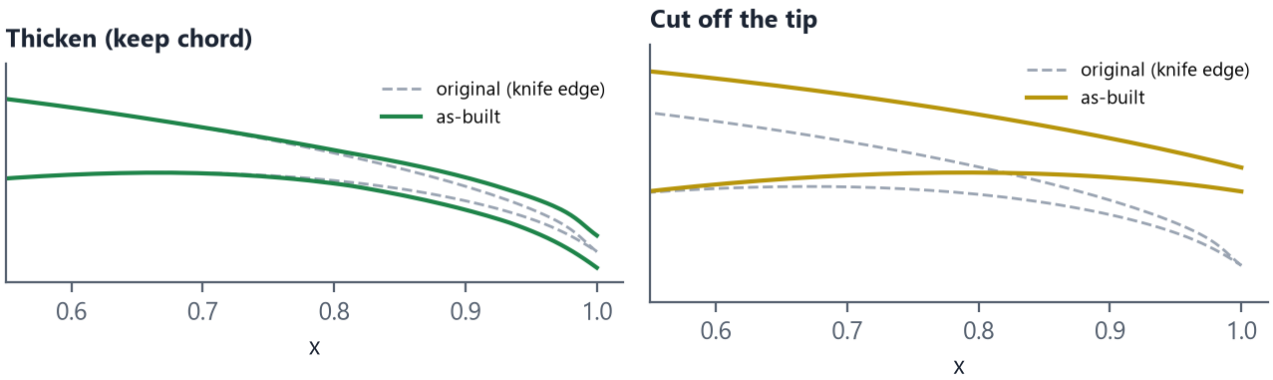


Figure 6 — The two trailing-edge treatments applied to a real section, zoomed on the aft. Thicken adds material over the rear while keeping the chord; Cut off truncates the tip and rescales.

Thicken	Adds thickness over the rear of the section, keeping chord and camber. Costs a fraction of a percent of downforce at a typical 1–1.5 mm trailing edge (about 0.3 % on the baseline section), rising to roughly 2 % at 3 mm. This is the default and the recommended aerodynamic preparation.
Cut off	Truncates the section where it reaches the target thickness and rescales to the original chord — for when a part or mold must literally be cut back. Thin-tailed sections lose far more downforce than the thicken treatment — roughly 17 % on the baseline section at 1.5 mm — and the tooltip says so.
Min thickness	An optional buildable-minimum check on the whole section. It warns when an element's thickest point is below the minimum and keeps thinner airfoils out of the optimizer and screener — it never silently reshapes a section you chose.

Thicken does more than open the trailing edge: it also raises the thickness of any station behind the section's nose that would otherwise be thinner than the trailing edge itself — a thin sail-like section would otherwise keep an unbuildable waist ahead of its blunt tail. A section whose thickest point is not meaningfully above the requested trailing edge is flagged **critically as a plate**: the treatment still opens and floors it, but the result is a near-constant-thickness plate the tool warns you about rather than a section you would have chosen (Figure 7).

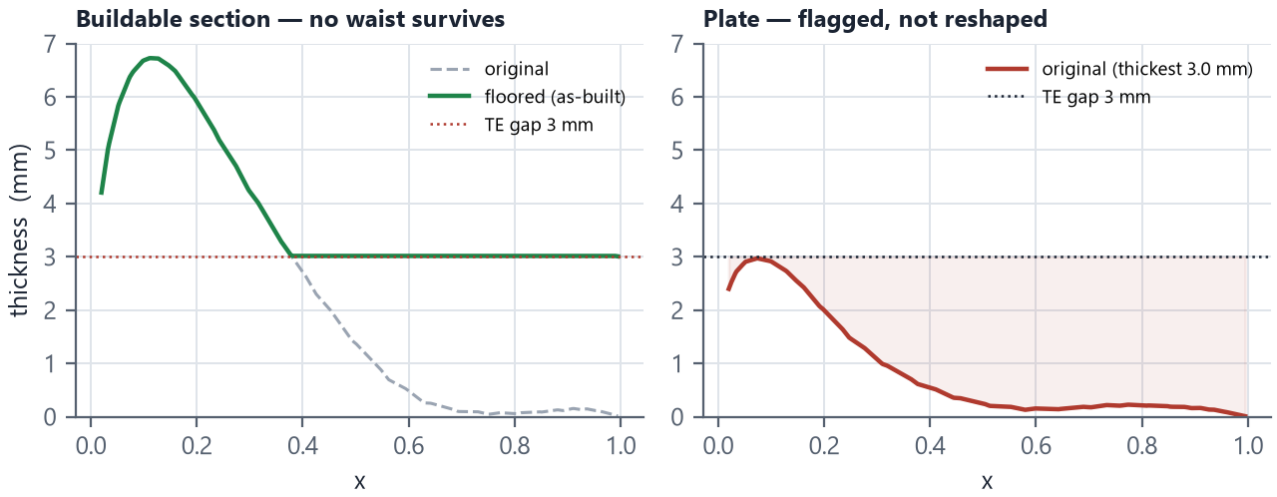


Figure 7 — As-built thickness distributions. Left: a buildable section, floored so nothing behind the nose is thinner than the 3 mm trailing edge. Right: a section thinner than the requested edge everywhere — a plate: still treated, but flagged critically.

> **Manufacturing preparation is fully reversible**
>
> Turning the checkbox on never alters your configuration — the treated shapes are derived on the fly, and turning it off returns the exact original geometry. Each element card shows the as-built trailing-edge thickness, maximum thickness, and the thinnest station behind the nose (the **waist** badge). Every downstream step — the solver, polars, the optimizer, and all exports — uses the as-built shapes, so what you cut is what was analyzed. If a design is destined for manufacture, keep the box on while optimizing so the search tunes the real part.

9 Support tools — screener and polars

Airfoil screener

The screener sweeps all bundled sections (plus any uploads) at a chosen Reynolds number and tabulates maximum lift, best efficiency, drag at a reference lift coefficient, thickness, camber, and the surrogate model's confidence. Columns sort on click; **Use** assigns a section to an element. Screening the full library takes a few seconds and is cached, so re-sorting and re-screening are instant. When manufacturing preparation is on, the thickness filter is pre-set so sections too thin to build are excluded up front.

Polars

The polar tab plots each element's lift curve and drag polar from the neural-network surrogate (trained on XFOIL data; instant), for the section as actually built. An **Add XFOIL reference** button runs the locally installed XFOIL executable, when present, for an independent cross-check — the two typically agree within a few percent below stall.

10 Step 6 — Export to CAD

DXF	True-scale millimeters, one closed contour per element on its own layer, as smooth splines (for lofting) or exact polylines. Opens directly as a sketch in SolidWorks, Fusion, Onshape, NX.
Complete bundle (ZIP)	DXF in both orientations, per-element XYZ point files, Selig .dat contours (main-chord units, for meshing and panel workflows), CSV, an SVG drawing, and a JSON manifest with the full configuration and analysis snapshot.
SVG drawing	True-scale vector drawing with the ground line, for documentation or 1:1 print checks.
Point table (CSV)	Every contour point in millimeters with element metadata.

OpenFOAM case A complete, ready-to-run 2D RANS case — mesh, boundary conditions, solver settings, and a run script — at a chosen mesh resolution. The handoff workflow is section 11.

Exports can be written in the as-driven orientation (ground at y = 0, ready for CFD domains) or upright (the CAD sketch convention). Each export is saved to the project's `exports/` folder with a timestamped name, and a dialog shows the exact location with a **Show in folder** button; a browser download of a copy is offered alongside. With manufacturing preparation enabled, every exported contour is the as-built profile, and the manifest records the achieved trailing-edge thickness per element.

The DXF export can also add **bounding-box lines**: four lines on a dedicated HITBOX layer — horizontals touching the stack's lowest and highest points, verticals at its extremes. Instant overall dimensions in CAD, deleted in one action with the layer.

11 Operating maps and RANS verification

Two closing tools round out the workflow. The operating maps show how a finished design behaves away from its design point — a wing tuned at one ride height rides at many — and the RANS tools hand a shortlisted section to a genuine Navier–Stokes solver for confirmation, either with one click inside the application or as an exported case.

Operating maps

The **Maps** tab (beside the Airfoil screener) sweeps the current design across ride height or speed — by default 10 to 120 mm of ride height in 23 steps, up to 40 points — and plots downforce and lift-to-drag against the swept value, marking the current operating point and the downforce peak. A ride-height sweep re-solves the flow at every point and takes a second or two at the sweep's capped panel resolution (reported under the charts; **Analyze** always runs at full resolution). A speed sweep is near-instant: the inviscid solution does not depend on speed, so only the Reynolds numbers and the dynamic pressure vary.

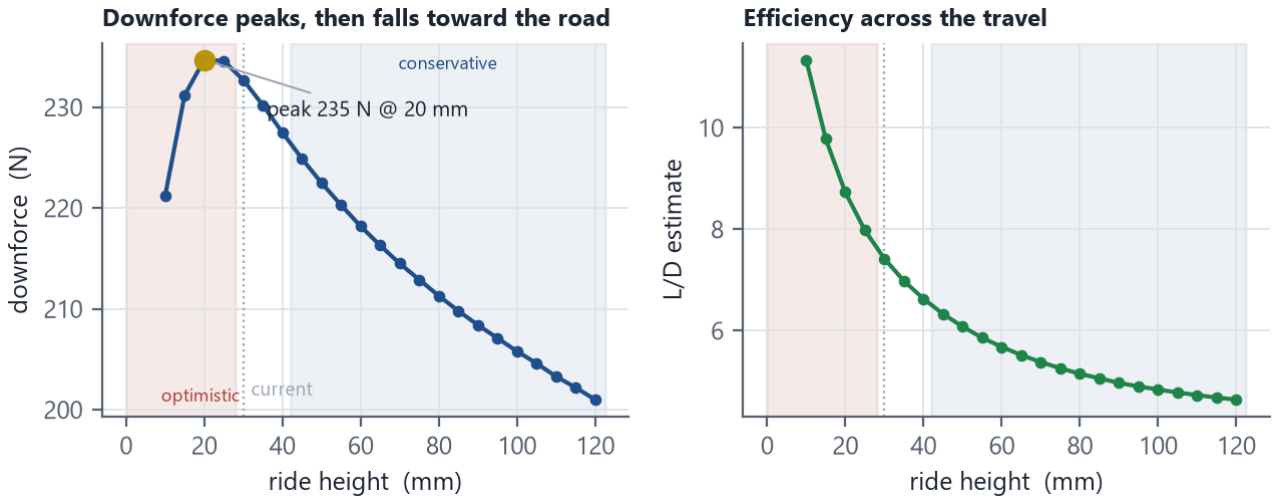


Figure 8 — The Maps tab's ride-height sweep of the default template, generated by the same sweep the application runs. Downforce peaks near 20 mm and falls closer to the road; the dotted line is the configured operating point.

The force-reduction peak the correction model predicts (section 14) is now directly visible instead of being a property taken on trust: on the default two-element template the peak sits near 20 mm ride height — running lower loses downforce. Swept points that push an element past twice its isolated stall limit are flagged beneath the charts: the model is optimistic there, so treat those downforce values as upper bounds.

Ride-height sweeps also shade the model's measured validity bands, from the RANS cross-referencing campaign recorded in the repository's `docs/calibration`: a red band below the venturi-choke onset (h/c ≈ 0.08), where fine-mesh truth runs measured the estimate optimistic and worsening toward the ground, and a blue band through the mid heights (h/c 0.12–0.35), where the same runs measured it conservative — the wing makes more than claimed there. Between the bands, at ordinary racing ride heights, the estimate was

validated to within a percent on the baseline section, and the choke boundary held across speeds. One caveat, also measured: the bands describe designs inside the loading budget. Configurations past the 90 % loading warning line measured 23–30 % optimistic even at the validated heights — when the loading meters warn, believe them over the bands.

The RANS case

Everything this tool reports screens and ranks; RANS decides. Both verification paths use the same complete, two-dimensional case built from the current section: a mesh with boundary layers resolved down to the wall on the wing surfaces, the steady k–ω SST setup, a ground plane moving at the freestream speed (the road, in the wing's frame), and automatic force-coefficient extraction. The reported lift coefficient is downforce-positive and referenced to the main chord — directly comparable to the coefficients this guide has used throughout. Its drag coefficient is the section's profile drag only: induced drag is a three-dimensional effect a section case cannot see, so it is compared against the estimate's profile share, never the total.

Coarse (~15k cells)	A first look — does the flow stay attached, is the case healthy. Converges in about a minute, but it is a screening number only: calibration testing measured the coarse mesh reading validated operating points 22–35 % below fine-mesh truth at racing and mid ride heights (the venturi gap is under-resolved), and the result says so.
Medium (~40k cells)	The standard confirmation run for a shortlisted design; typically a few minutes. Agreed with the fine mesh within the oscillation band at the calibration anchor.
Fine (~90k cells)	Final numbers, calibration-grade comparisons, and the grid-sensitivity check for a design you intend to commit to. Heavily loaded sections can need many thousands of iterations here — let it run.

In-app verification — the RANS verify tab

The **RANS verify** tab runs the case without leaving the application. It needs Docker Desktop running (the button says so when it is not; the first run downloads the official OpenFOAM image, about a gigabyte, once). Press **Verify with RANS**: the tab meshes the current section, starts the solver in a container, and streams live convergence — iteration count, lift and drag, and the lift history charted against the panel estimate. The run stops itself as soon as the force history is statistically flat (or the solver's residuals converge first), so the generous iteration cap is an upper bound, not a duration.

A finished run reports the RANS coefficients — the mean over the settled tail of the history, with the residual oscillation as a ± band — beside the panel estimate, the equivalent downforce at the wing's reference area, and the drag comparison. If the run hit its iteration cap while the lift was still trending, the result is labelled **NOT CONVERGED** in plain terms: the numbers are a mid-transient snapshot, and the remedy — raise the cap and rerun — is stated with it.

Two more things arrive with the result. First, the **suggested ground-gain factor**: the pinned calibration value that would make the estimate reproduce the RANS sectional lift at this operating point, applied to the configuration with one click — the calibration loop of section 14, closed with real data. It is only offered from a converged run, and a coarse-mesh result carries a caution beside it: pin a factor from a medium or fine run, never from coarse, or the mesh's own bias becomes the session's calibration. Second, the **flow field**: the solved section flow rendered in the same as-driven view as the drawing — velocity magnitude with streamlines, or pressure coefficient — which shows at a glance what the coefficients cannot: whether the slots are flowing, where the venturi works, and whether the flap system is separated (the usual reason RANS and the attached-flow screening estimate disagree on an aggressive design).

The exported case (WSL)

The **OpenFOAM case** card in the Export tab writes the same case into the exports folder for manual runs, other machines, or post-processing in ParaView. Run it from the case folder under WSL — the one-time OpenFOAM installation is described in the repository README:

```
wsl -d Ubuntu -- bash run.sh
```

The script prepares the OpenFOAM environment, converts and checks the mesh, runs the solver, and writes the final coefficients to `results.txt`; each case's `README.txt` records its exact numbers and conventions. The in-app and WSL paths execute the identical script and agree with each other; compare either result with the estimate here and re-tune the calibration factors (section 14) against it — closing exactly the loop those factors were built for.

PART II

How it works

Stopping a run without losing it

Stop & keep fields ends a running solve gracefully: the solver writes its current fields first, so the velocity and pressure views work on the partial run. The verdict is labeled "stopped by user" and no k_g suggestion is offered — a hand-stopped force history, however flat it happens to look, is a preview, not a calibration point. Cancel remains the hard stop that keeps nothing.

12 Architecture

The application is one local process. A thin desktop shell (a WebView2 window) hosts a browser user interface built from plain HTML, CSS, and JavaScript with no build step. That interface talks to an in-process web server over a loopback-only HTTP connection; the server exposes every capability as a small REST API and delegates all real work to a framework-free Python core.

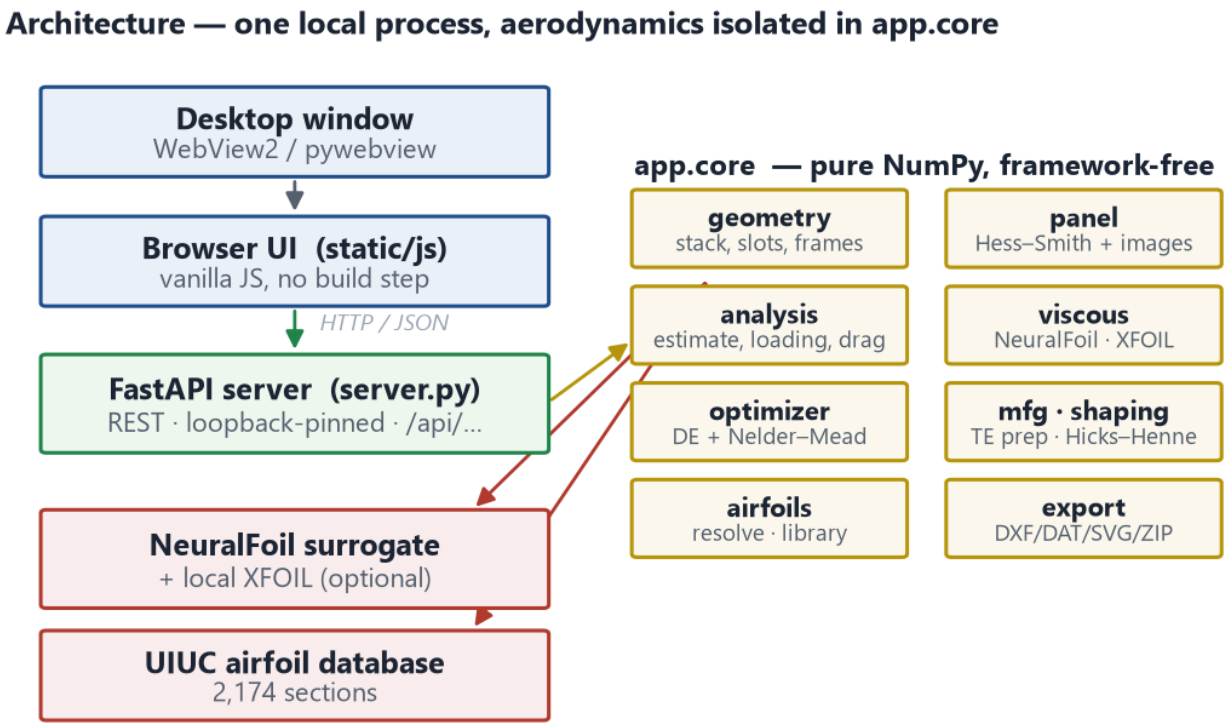


Figure 9 — The whole application in one process. All aerodynamics lives in app.core, framework-free Python, reachable identically from the window or from scripts.

Everything aerodynamic lives in `app.core`: the panel solver is pure NumPy, with NeuralFoil and AeroSandbox for viscous data and geometry, SciPy for the optimizer's global search, and Matplotlib and ezdxf for geometry tests and DXF output. No web framework reaches into it. The web layer is a wrapper: the same functions that draw the live section and run the optimizer are importable directly for scripting, and the regression suites exercise them without a server. The server binds only to the loopback address and rejects any request whose Host header is not local, closing the one route by which a web page could otherwise reach it.

Frames of reference

Two coordinate frames recur throughout. The **design frame** is upright — lift positive-up, the main leading edge at the origin, the main chord equal to one. The **installed frame** is the design frame flipped so downforce points down, with the ground plane at y = 0 and the whole section translated so its lowest point sits at the ride height. The panel solver runs in the installed frame; the two orientations you can draw and export are exactly these frames. All stack coefficients are referenced to the main chord; per-element lift coefficients are referenced

to each element's own chord.

13 The inviscid panel method

The core solver is a multi-element Hess–Smith panel method. Each element contour, in Selig ordering, is discretized into flat panels. Every panel carries a constant-strength source distribution (one unknown per panel), and all panels of a given element share a single vortex strength (one more unknown per element). Sources set the shape of the flow; the per-element circulation sets its lift.

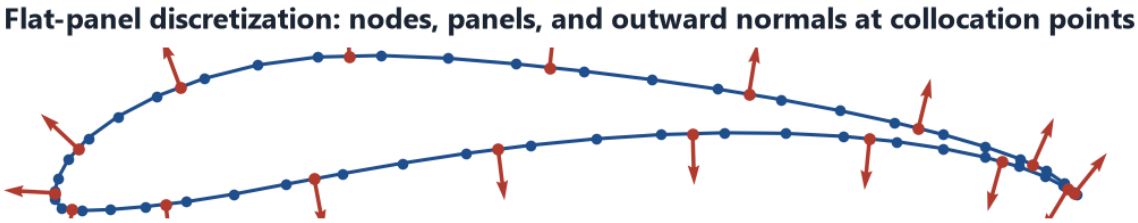


Figure 10 — A real section discretized into flat panels. The solver enforces the boundary conditions at the collocation point of each panel, using the outward normals shown.

The unknowns are fixed by two sets of conditions. At the collocation point of every panel, **flow tangency** requires zero velocity normal to the surface — the solid-wall condition. For each element, one **Kutta condition** requires the flow to leave the trailing edge smoothly, expressed as equal-magnitude tangential velocities on the two panels meeting there. Together these give a single dense linear system spanning all elements at once, so every slot and stack interaction is captured exactly — there is no assumption that the elements are independent.

Ground effect by images

The ground plane is introduced by the method of images: the velocity the real system induces at a point equals what a mirror system induces at the mirrored point, under a single reflection identity. Reflecting a source keeps its sign; reflecting a vortex flips it; both facts fall out of the same identity, so the ground adds no new unknowns — only an extra influence term. This is important because the naive alternative (a lift-from-circulation shortcut) is simply wrong near the ground, where image-induced velocities are large.

Forces from pressure

Once the strengths are known, the surface pressure coefficient at each panel follows from its tangential velocity, and the forces and moment come from **integrating that pressure** over the surface — not from the circulation. Pressure integration remains valid in ground effect, where the circulation-based shortcut over-predicts downforce by roughly a factor of two because it ignores the image-induced velocity field. The moment is reported nose-up-positive, and the centre of pressure follows from it. A small numerical drag residual (exactly zero for ideal inviscid flow) is reported as a discretization diagnostic.

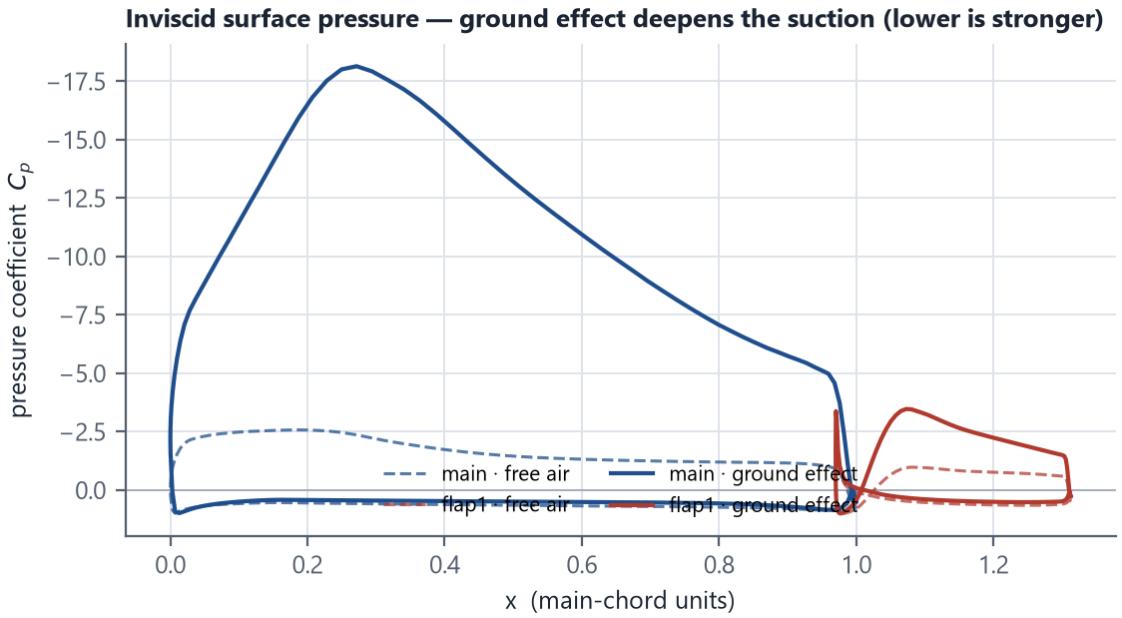


Figure 11 — Real inviscid surface pressure for a two-element stack, free air (dashed) versus ground effect (solid). Ground effect deepens the suction on the main element's lower surface — the venturi that makes downforce.

14 Ground effect and the correction model

The panel solution is exact for inviscid flow, but inviscid ground effect is unphysical in the limit: as the ride height shrinks, the predicted venturi gain grows without bound, while a real flow chokes on boundary-layer growth in the gap. The tool therefore reports a corrected estimate built from transparent, user-adjustable factors:

```
C_est  = eta_visc · [ C_free + G_real ]
G_real = cap · tanh( k_g · (C_ground − C_free) / cap ) · tanh( (h/c) / 0.045 )
cap    = 3.0 · |C_free|
```

Here `C_free` and `C_ground` are the exact inviscid downforce coefficients in free air and in ground effect. `eta_visc` (default 0.85) is the free-air viscous realization — how much of the inviscid free-air load a real viscous flow achieves. `k_g` is the fraction of the additional ground-effect gain that is realized; at ordinary ride heights both saturation terms are near-linear and near-one, so the model reduces to the familiar `eta_visc · [C_free + k_g · (C_ground − C_free)]`. The first saturation caps the realized gain at three times the free-air load; the second chokes it off below h/c ≈ 0.045, where the real venturi flow stalls on boundary-layer growth. Left blank, `k_g` follows an automatic ride-height curve:

```
k_g = 0.85 · tanh( (1.4 / 0.85) · h/c )
```

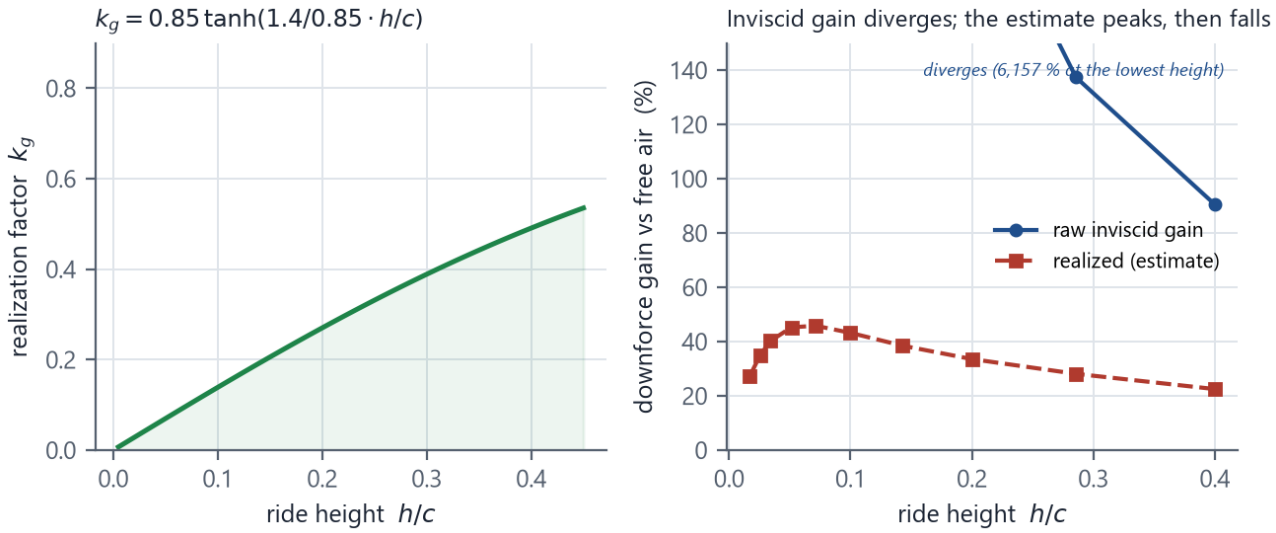


Figure 12 — Left: the automatic realization-factor curve, which preserves a calibrated small-height slope and an 0.85 ceiling but vanishes at the ground. Right: on a real stack, the raw inviscid gain diverges as ride height shrinks while the realized estimate peaks and then falls toward the road.

Together the saturations keep the calibrated slope at racing ride heights while genuinely reversing at the bottom of the travel: the estimate peaks near h/c ≈ 0.065 on the two-element baseline and decreases as the wing drops further — the force-reduction behaviour measured on wings in strong ground effect — instead of growing without bound. Both knobs are exposed in the calibration group beside the raw inviscid numbers they modify: `eta_visc` directly, and `k_g`, which follows the automatic curve when left blank and is pinned when a value is entered — useful for matching RANS or tunnel data at a known ride height. The analysis always reports which source it used. The cap ratio and choke height (`gain_cap_ratio`, `choke_h_c`) are configuration fields with conservative defaults, adjustable at the config/API level.

15 Viscous section data

The panel method knows nothing of viscosity, so the section-level viscous quantities — maximum lift, profile drag, the angle for a given lift — come from NeuralFoil, a neural-network surrogate trained on large sweeps of XFOIL solutions. It evaluates a complete polar in milliseconds, which is what makes library-wide screening and thousands of optimizer evaluations feasible, and it reports a **confidence** value that falls for unusual sections and operating points.

A locally installed XFOIL executable — dropped into the application's `xfoil/` folder; it is not redistributed with the tool — serves as an optional independent cross-check through the same interface; it is slower and single-element, but it is the reference the surrogate approximates. Polars are cached by section, Reynolds number (bucketed to three significant figures), transition criterion, and model size, so a repeated operating point is free. The optimizer and the final analysis read the same fast surrogate; the polar tab reads a larger, more accurate one. The optimizer's speed comes instead from coarser panel resolution during the search.

> **Where to be sceptical**
>
> The surrogate is most trustworthy for conventional sections at moderate Reynolds numbers. Very unusual shapes or very low Reynolds numbers deserve the XFOIL cross-check. When a section's polar never stalls within the analyzed angle range, the reported maximum lift is a lower bound rather than a true stall value, and the loading warnings say so.

16 Element loading budget

A slotted high-lift system can carry more total lift than the sum of its isolated elements, but each element still has a viscous ceiling. Following classical high-lift practice, the tool budgets each element's **free-air** inviscid load

— knocked down by the viscous realization factor — against that element's isolated maximum lift coefficient, evaluated at its own Reynolds number (Figure 3). Warnings begin at 90 % of the limit and criticals at 110 %, since slotted elements tolerate somewhat more than isolated maximum.

Free-air load is used for that budget deliberately: raw inviscid ground-effect loads grow without bound near the road and would make the check meaningless. A second indicator watches each element's **realized ground-effect operating point** — the same lift coefficient the profile-drag lookup uses, so it is consistent with the reported estimate by construction. Sections near the ground have been measured carrying roughly up to twice their isolated maximum before the flow gives up, so an element past that allowance draws a screening-validity warning: treat the downforce estimate as optimistic there, and the profile drag — capped at the pre-stall polar — as understated. Each element's inviscid ground-to-free multiplier is reported alongside, so the loading picture stays honest while the ground gain remains visible.

17 The drag model

Reported drag has two parts. **Profile drag** is each element's own section drag, read from its polar at the lift coefficient it actually works at in ground effect (its free-air load plus the realized share of its ground-effect gain, viscous-realized), then summed over the elements weighted by chord. **Induced drag** is computed from the very downforce the tool reports, so lift and induced drag are always consistent:

```
CD_i = C_L² / (pi · AR · e) · phi
```

where `AR` is the wing aspect ratio, `e` the span-efficiency input (endplates push it toward one), and `phi` the McCormick ground-effect factor evaluated at the section's mid-height — wings near the ground shed much weaker trailing vorticity, so induced drag falls as the road approaches:

```
phi = (16 h/b)² / (1 + (16 h/b)²)
```

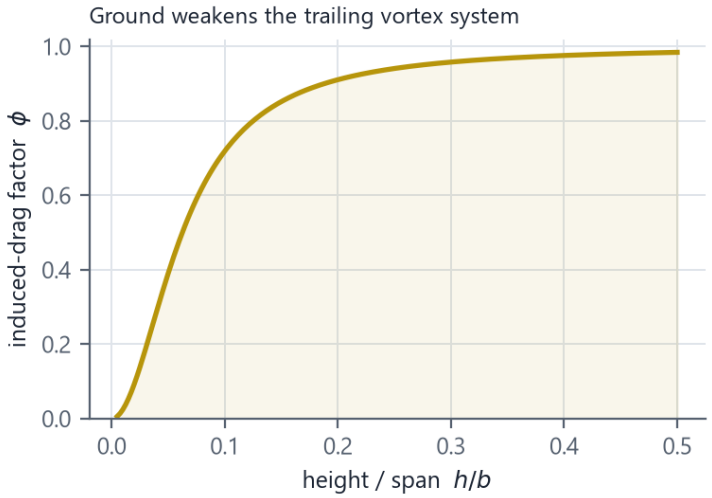


Figure 13 — The McCormick factor. Close to the ground, the trailing vortex system weakens and induced drag drops sharply.

For a loaded finite-span front wing, induced drag dominates the total — often by an order of magnitude over profile drag — so leaving it out (as a profile-only figure would) produces fantasy efficiency numbers. Interference and mounting drag are not modeled.

18 The airfoil spec system

Every airfoil in the application — a library section, an upload, a manufactured section, a shape-modified section — is named by a single **spec string**, and one resolver turns any spec into coordinates. Modifiers are prefixes that wrap a base spec, so they compose:

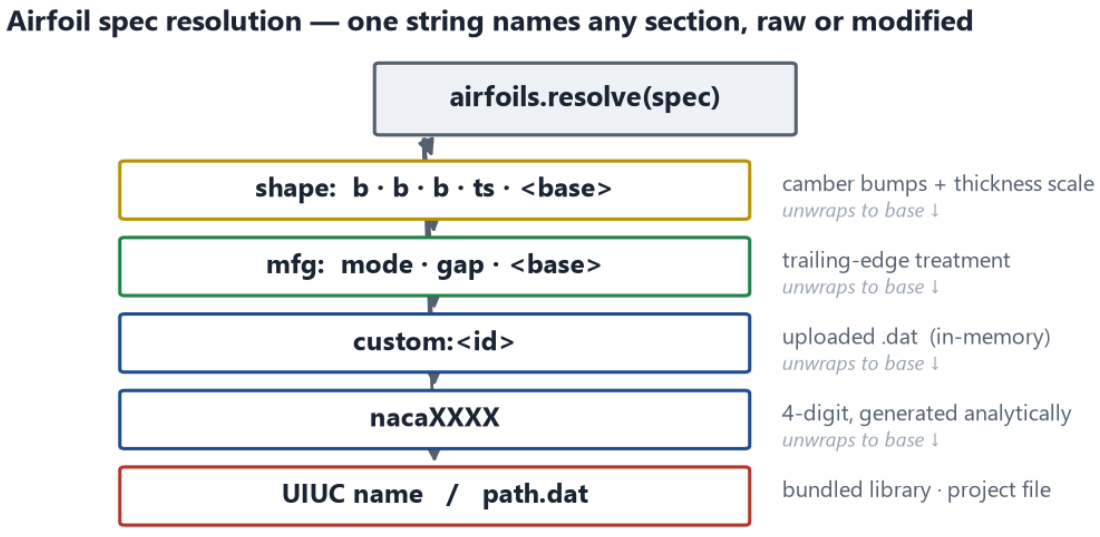


Figure 14 — Spec resolution. Each modifier unwraps to its base and re-applies its transformation, so a shaped, manufactured library section is one string that every part of the system understands.

Because resolution is centralized, a modified section is a first-class airfoil everywhere with no special cases: the panel solver, the NeuralFoil polars, the XFOIL cross-check, the screener, and every exporter all see the same as-built coordinates. Modifier parameters are quantized (trailing-edge gaps and shape parameters each snapped to a fixed grid) so the spec strings — and the repanel and polar caches keyed on them — stay stable while the optimizer sweeps continuous values.

19 Manufacturing geometry

The **thicken** treatment adds thickness symmetrically about the camber line, blended in over the rear of the section with a smooth ramp, so the added opening between the two surfaces at the trailing edge equals the requested gap exactly. It then applies a **thickness floor**: from the first station where the section reaches the requested gap, back to the trailing edge, no station is left thinner than the gap. This is what removes the mid-chord waist that a naive ramp leaves on thin sections (Figure 7). The treatment works on a densely repaneled copy of the base section, and the floor is guaranteed on the geometry every consumer actually sees: the floored contour is verified through a spline-repanel round-trip, with the floor raised adaptively until it survives the fit.

The **cut off** treatment truncates the section at the station where it reaches the target thickness — solved so that after rescaling to the original chord the trailing edge is exactly the requested thickness — and closes it with a flat base. It applies no floor, because its purpose is to cut a real part back; instead the report measures any remaining waist and warns. Because camber is preserved by the symmetric construction, the treatment is a **manufacturing operation, not a hidden aerodynamic re-trim**: shifting material onto one surface instead would just be a camber change, which the optimizer's real variables already control.

Exports close the blunt base explicitly. In the spline DXF the base is a separate straight line between the two surface endpoints, keeping the corners sharp instead of rounding them; the polyline DXF and the point files carry the exact open contour. The manifest records the achieved trailing-edge thickness, maximum thickness, and the measured minimum thickness behind the nose for every element.

20 Shape refinement mathematics

Shape refinement modifies a section with a small, smooth, airfoil-preserving parameterization. The camber line is displaced by a sum of three **Hicks–Henne bump functions** centred near 25, 55, and 80 % chord — each a

smooth hump that is zero at both ends of the chord and peaks at its centre — and the thickness envelope (each surface's offset from the camber line) is scaled by a single factor. Four numbers per element (Figure 5).

This choice is deliberate. Four smooth parameters keep the added search dimensions cheap and keep every generated shape recognizably an airfoil, so the NeuralFoil surrogate stays inside the space it was trained on and its confidence remains meaningful — a free-form coordinate parameterization would wander into shapes the surrogate cannot rate. The shape wraps as a spec modifier (section 18), so manufacturing preparation applies on top of the shaped section, and the buildable-thickness minimum limits how far the thickness scale may thin an element. Re-optimizing a shaped design unwraps and seeds from its current parameters rather than nesting a second modification.

21 The optimizer

The optimizer treats the design as a vector of continuous variables and minimizes a single objective. Continuous quantities — stack angle, deflections, slot gap and overlap, chords, shape parameters — map directly. **Airfoil choice** is encoded as one continuous variable per element mapped onto a shortlist of candidate sections ranked by maximum lift, so that neighbouring values are similar airfoils and the search stays meaningful. The shortlist is built by the screener at each element's Reynolds number, includes the currently selected section (unless the candidate pool is restricted to your uploads), and — when manufacturing sets a buildable minimum — excludes sections too thin to make at that element's chord.

Objective and constraints

The objective is dominated by the squared relative error to the target downforce, so the search first and foremost hits the number you asked for. Added to it are a weighted drag term (the adjustable drag penalty, trading residual drag against exactness on target) and soft penalties that keep candidates inside the healthy high-lift envelope: slot gap and overlap bands, element loading below about 105 % of each airfoil's isolated maximum, and a hard penalty on intersecting geometry or a failed solve. The loading penalty is weighted so that an honest miss of the target beats a design that only reaches it by driving an element deep into stall.

A further soft term charges for viscous-data trust: each evaluation already computes the surrogate's own confidence and whether a loaded element sits at the edge of its polar grid or on a clamped drag lookup, and the objective now uses them — quadratic growth as confidence falls below the screener's 0.5 bar, plus a fixed bump per loaded element on edge data. The penalty is baseline-relative like the loading bands (a re-optimized shaped design pays only for leaning harder on low-trust data) and sized so a fully flagged element costs about an 11 % target miss — enough to prefer an equally-performing trustworthy design, never enough to beat hitting the target.

Every soft band is widened once per run so the starting design's own operating point is penalty-free, and the default search bounds stretch to include its variable values. The reason is a do-no-harm contract: the analysis page reports an aggressive design's downforce as the headline number, so an optimizer that refuses that same operating point could only ever return less than the user already has. From a healthy start nothing changes — the absolute envelope still bars the search from wandering into separated-flow loading; from a hot start the penalty punishes getting hotter than the baseline, not matching it. Bounds set explicitly through the API, and the manufacturing thickness floor, remain hard limits.

Search, candidates, and reproducibility

The search is a seeded global phase (differential evolution) followed by a local refinement (Nelder–Mead), evaluating a few hundred to a few thousand complete analyses. Identical inputs give identical results — the search is fully seeded. Every feasible evaluation is archived; at the end, the best design plus a few maximally-different on-target designs are selected from that archive as the candidate set, each re-analyzed at full resolution.

Fast and Standard runs stop as soon as the target is met, which is quick but means the design depends on which basin the search reached first. Thorough runs remove that dependence: the global phase runs to

completion with a wider population, and the final polish runs from several diverse starting points at full panel resolution, with the reported best required to come from full-resolution evaluations. The result is that two Thorough runs from different starting configurations converge to the same lowest-drag design, where Fast runs might differ.

22 Numerical accuracy and validation

The panel method is validated against an independent linear-vorticity formulation (with forces taken by pressure integration) and against XFOIL's inviscid solution on single elements, agreeing to within a few percent in free air and in ground effect. The image system is checked against an explicitly modelled mirror geometry, and the slot gap/overlap solver against direct geometric measurement of the placed sections.

The repository ships regression suites that exercise the solver physics, the manufacturing geometry (including the thickness floor and plate detection), the optimizer (including determinism, Thorough reproducibility, and the do-no-harm baseline contract), the RANS runner (convergence verdicts, calibration gating, the flow-field pipeline), the shape system, and the API's failure behaviour under hostile input. They run from a single command and form the contract the application is held to; every figure in Part II of this guide is generated from the same core the suites test. The RANS case itself is cross-checked two ways: the identical case solved in the container and under WSL reports the same coefficients.

Beyond the suites, the estimate was cross-referenced against its own truth model in a logged RANS campaign — ride-height sweeps at three mesh fidelities plus optimizer winners, every run recorded with its verdict in the repository's `docs/calibration`. That campaign is where the validity bands on the operating map, the sub-choke warning, the coarse-mesh caution, and the candidate trust-badge validation all come from — each number in the interface traces to a run in that log.

> **What the numbers are for**
>
> Treat every downforce and drag figure as a screening and ranking value. The tool is calibrated to land a typical two-element section at racing ride heights in the right physical band, but it resolves neither the boundary layer, slot merging and separation, nor three-dimensional effects. Confirm a shortlisted design with RANS CFD or measurement, and re-tune the calibration factors against those results, before committing to it.

23 Model scope and limitations

- **Two-dimensional section physics** with classical finite-span corrections. Spanwise design — endplates, footplates, local twist — is out of scope.
- **Inviscid pressure field.** Boundary-layer displacement, slot merging, and separation are represented only through the calibrated corrections and the loading limits, not resolved.
- **Calibrated ground effect, with a measured validity map.** The correction factors are tuned, not derived. A RANS cross-referencing campaign (recorded in the repository's `docs/calibration`) validated the estimate on the baseline section at ordinary racing heights and at large gaps, measured it optimistic below the venturi-choke onset (a standing warning fires below h/c 0.08) and conservative through the mid-height gain peak — single-section, fully-turbulent 2D evidence; tunnel data remains the referee.
- **Surrogate viscous data.** Section polars come from a neural surrogate of XFOIL; its confidence is reported and a cross-check is one click away, but unusual sections at low Reynolds numbers deserve scrutiny.
- **No interference or mounting drag**, and no compressibility (irrelevant at these speeds).

24 Reference

Project files	Save and Open write a portable JSON project containing the full configuration, target, and any uploaded airfoil geometry (including inside shape and manufacturing wrappers). The working session is also autosaved server-side, in app_data/ beside the application, and restored on launch.
Exports folder	Exports are written to exports/ beside the application, with timestamped names; the export dialog links to the exact file.
Automation / API	Every function is available over the local REST API; interactive documentation is served at /api/docs while the application or the headless server is running.
Headless use	The same server runs without a window for scripting: python -m uvicorn app.server:app --port 8642.
Verification	The regression suites run from a single command: python app/tests/run_all.py (it starts its own scratch server for the API checks).
Design record	Every design decision behind the application is recorded, with the options considered, in DECISIONS.md at the repository root.